

Chapter 4

Primitive Types, Operators, and Strings

There are several respects in which Java treats integers differently from types like `FilledRect` and `Location`. One example is the means Java provides to describe a specific piece of information. For most types, we have used constructions. To describe the `Color` light blue, for example, we might say:

```
new Color( 255, 100, 100)
```

and to describe the origin of the coordinate system we say:

```
new Location( 0, 0)
```

For integers, on the other hand, Java provides us with the ability to write literals that directly describe the desired value. A programmer would simply write

```
3
```

rather than

```
new int( 3)
```

to describe the integer value 3 within a program.

Another distinction between `ints` and many of the other types is that there are no methods associated with `ints`. We perform operations on `ints` using operator symbols like `+` and `*`, while we use method invocations when we want to perform operations on `Locations` or `FilledRects`.

These differences are partly a matter of convenience. Numbers are used so frequently in programs that it is important to provide convenient mechanisms for manipulating numeric information. At the same time, they represent a more fundamental aspect of the way Java views integers. In Java's view, integers are in some sense less transient than the other kinds of information we have considered. Java doesn't let us say

```
new int( 3)
```

because 3 already exists before you use it. It would make no sense to talk about making a new 3. How would the new 3 differ from the old 3?

The integers are not the only Java type that has these special properties. The type `boolean` shares these features with the integers. There are no methods to manipulate `booleans`, and programmers use the literals `true` and `false` rather than constructions to describe specific `boolean` values.

In general, Java refers to pieces of information that we construct using `new` and can manipulate with methods as *objects* and the types composed of such items are called *object types* or *classes*. Pieces of information that are described using literals and manipulated with operators rather than methods are called *primitive values* and types composed of primitive values are called *primitive types*.

In this chapter, we will more carefully explore the use of the two primitive types we have already encountered, `int` and `boolean`, and then introduce some additional primitive types. In particular, we will learn about the type `double` which is used when working with numbers that are not integers.

Finally, we will examine a few aspects of the type `String`. All the examples of quoted text we have seen in the preceding chapters are literals that describe elements of the `String` type. The `String` type is an object type rather than a primitive type. In some ways, however, it resembles a primitive type. Java provides both operators and methods to manipulate `Strings`, and it is the only object type in Java whose objects can be described by literals rather than only by constructions.

4.1 Operators versus Method Invocations

As observed in the introduction, there are no methods associated with primitive types. They can only be manipulated using operators. This reflects the fact that in Java, values of primitive types are permanent and unchanging.

Operators produce new values rather than modifying existing values. Thus, when we evaluate the expression

```
count + 1
```

`count` does not change. Instead, we produce a new value separate from `count`. If we want to change `count`, we could incorporate this expression in an assignment statement such as

```
count = count + 1;
```

but in this case it is the assignment rather than the arithmetic operation that is changing `count`.

On the other hand, methods may modify existing objects. If we say

```
box.move(1,1);
```

then `box` changes. No new object is produced.

To better understand this difference, consider the impact of a small change we might make in the grid drawing program presented in Figure 1.10. Recall that each time the mouse is clicked, this program draws thin vertical and horizontal rectangles. The locations of the corners of the rectangles are changed with each click so that after several clicks, the pattern drawn forms a grid as shown in Figure 4.1.

The positioning of the rectangles drawn with each click is controlled by two `Location` variables named `verticalCorner` and `horizontalCorner`. In the original version of the program, these variables are initialized by the assignments

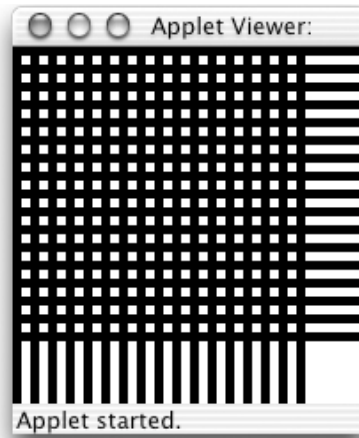


Figure 4.1: Pattern construct by `DrawGrid`

```
horizontalCorner = new Location(0,0);
verticalCorner = new Location(0,0);
```

They are then updated with each click by the invocations

```
verticalCorner.translate(10,0);
horizontalCorner.translate(0,10);
```

found in the `onMouseClicked` method.

Each of the two assignment statements that initialize these variables contains a construction that creates a new `Location` object. In other contexts, we have suggested that, when possible, one should reduce the amount of work required to execute a program by reusing an existing object rather than creating a new object. This suggests that it might be better to replace the assignments used to initialize these variables in the original program with the assignments

```
horizontalCorner = new Location(0,0);
verticalCorner = horizontalCorner;
```

Surprisingly, making this change would change the behavior of the program considerably. Instead of drawing a grid as shown in Figure 4.1 it would produce the pattern shown in Figure 4.2.

In the original code, the names `horizontalCorner` and `verticalCorner` are initialized to refer to two different objects that represent the same point on the `canvas`. Using the revised code, there would actually only be a *single* `Location` object created. Both names would refer to the same `Location` object. This seemingly subtle distinction produces a significant change in the program's behavior because if the information associated with an object is changed by a mutator method, the change is visible through all names associated with the object.

The names `horizontalCorner` and `verticalCorner` still both refer to (0,0) when the revised program first responds to a mouse click, just as they did in the original program. As a result, the first two rectangles drawn are identical to those drawn by the original program. The effect of executing the last two statements found in `onMouseClicked`

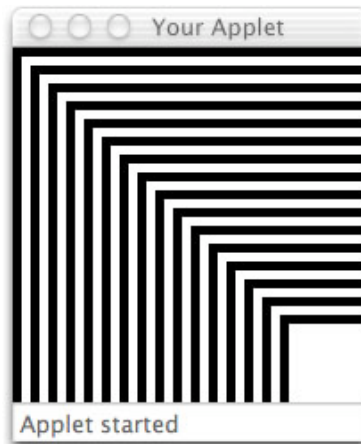


Figure 4.2: Pattern drawn by DrawGrid program after modifications

```
verticalCorner.translate(10,0);
horizontalCorner.translate(0,10);
```

is, however, quite different in the modified version. In the original program, the intent of these commands is to move the point referred to by `verticalCorner` a bit to the right and to move the point referred to by `horizontalCorner` a bit down the screen. After the first execution of these statements in the original program, `verticalCorner` refers to the point (10,0) and `horizontalCorner` refers to the point (0,10). In the revised program, both names refer to the same object. When the object named `verticalCorner` is moved to the right, the object named `horizontalCorner` must also move to the right. So after the two invocations of `translate` are complete, both names still refer to the same `Location` object, and this object describes the point (10,10).

In the original program, the points associated with `horizontalCorner` and `verticalCorner` get farther apart each time the mouse is clicked. In the revised version, they always refer to the same point on the screen. This point will move down the diagonal of the canvas as the mouse is clicked repeatedly. As a result, the corners of the rectangles drawn meet along this diagonal as shown in Figure 4.2.

By contrast, consider what happens if we perform as similar a sequence of operations as possible using `int` variables instead of `Locations`. A version of `DrawGrid` rewritten to use two `int` variables instead of two `Locations` is shown in Figure 4.3. The output of this program is identical to the original version of `DrawGrid` that used two separate `Location` objects. This version, however, uses one `int` named `verticalCornerX` to keep track of the x coordinate of the corner of the next vertical rectangle to be drawn and a second variable, `horizontalCornerY`, to keep track of the y coordinate of the horizontal rectangles. The version shown in the figure initializes the two `int` variables using the assignments

```
horizontalCornerY = 0;
verticalCornerX = 0;
```

Suppose we try to mimic the change we made in the version that used `Locations` by replacing these assignments by the statements shown below:

```

    // A program that draws a grid of thick black lines
    // on the canvas
public class DrawGrid extends WindowController {

    private static final int THICKNESS = 5;    // Thickness of FilledRects

    // Coordinates of the next two rectangles to draw
    private int verticalCornerX, horizontalCornerY;

    // Set coordinates to position first pair of rectangles at upper
    // left corner of the canvas
    public void begin() {
        horizontalCornerY = 0;
        verticalCornerX = 0;
    }

    // Draw a pair of rectangles and move coordinates so that the next
    // pair of rectangles will appear further down and to the right
    public void onMouseClick(Location point) {
        new FilledRect(verticalCorner, THICKNESS, canvas.getHeight(), canvas );
        new FilledRect(horizontalCorner, canvas.getWidth(), THICKNESS, canvas );

        verticalCornerX = verticalCornerX + 2*THICKNESS;
        horizontalCornerY = horizontalCornerY + 2*THICKNESS;
    }
}

```

Figure 4.3: Drawing a grid without Locations

```

horizontalCornerY = 0;
verticalCornerX = horizontalCornerY;

```

Would the program's output remain the same, change to look like the drawing in Figure 4.2, or would it be different from both of the versions that used `Locations`?

The answer is that the behavior of the program would be unchanged. The assignment

```
verticalCornerX = horizontalCornerY;
```

makes the two variables refer to the same value just as the assignment

```
verticalCorner = horizontalCorner;
```

makes the two variables refer to the same object. The difference is that applying a mutator method can change an object, but primitive values do not change.

The “+” operator does not change one number into another. When we execute the assignment statement

```
verticalCornerX = verticalCornerX + 2*THICKNESS;
```

for the first time, the value associated with `verticalCornerX` changes from 0 to 10. This is not because the addition operator turns 0 into 10. While the `translate` method may modify the object to which it is applied, the addition operator produces a new value without in any way modifying its operands. The assignment operation then associates the new value produced with the name `verticalCornerX`.

In general, changes made by an assignment to a numeric variable only affect the variable explicitly mentioned in the assignment. Changes made by applying a mutator method to an object, however, will be visible through all names associated with the object that is modified.

Exercise 4.1.1 *Suppose the following `Location` variables have been declared as instance variables and then initialized with these assignments:*

```
location1 = new Location( 56, 34 );
location2 = new Location( 34, 53 );
location3 = location1;
```

The following statements are then executed.

```
location1.translate( 22, 14 );
location2.translate( -23, 18 );
location3.translate( 15, -22);
```

What are the values of the following?

- a. `location1.getX()`
- b. `location2.getY()`
- c. `location3.getY()`

4.2 Separate but Equal

The `==` operator was introduced in Section 3.3 as a means to compare integer values. It actually can be used to compare values of any type. The only problem is that when used with object types it often returns an unexpected result. When applied to objects, `==` returns `true` only if both of its operands describe exactly the same object. That is, if two objects are created by separate constructions, then comparing them with the `==` operator will produce `false` as a result.

Recall the scenario discussed in the preceding section in which we considered the behavior of the two `Location` variables `verticalCorner` and `horizontalCorner` of the `DrawGrid` program. If, immediately after these variables were initialized by the assignments

```
horizontalCorner = new Location(0,0);
verticalCorner = horizontalCorner;
```

we evaluated the expression

```
verticalCorner == horizontalCorner
```

the result produced would be `true`. On the other hand, if the same equality test was evaluated after the pair of assignments used in the original version of the program:

```
horizontalCorner = new Location(0,0);
verticalCorner = new Location(0,0);
```

then the result would be `false`.

There are situations in which it is useful to test whether two variables refer to exactly the same object. In these situations, `==` is just what is needed. It is more common, however, to need to determine if two objects currently have the same characteristics. For example, suppose you want to write a simple program to test how steady your hand is as you click the mouse. The program we have in mind will display the word “STEADY” whenever you managed to click on exactly the same point on the screen twice in a row. Otherwise it would leave the screen blank.

Figure 4.4 shows an almost complete implementation of such a program. Each time the mouse is clicked, the `onMouseClicked` method checks to see if the current point is the same as the last point and shows a “STEADY” message if it is. To do this the program maintains a variable `lastPoint` that refers to the last point at which the mouse was clicked. This variable is initialized to refer to a point outside the canvas so that the person running the program will never appear to succeed on the first click.

The `if` statement in `onMouseClicked` is incomplete in the figure. To complete the program, we need to add a condition that will test to see if the last point and the current point are the same.

The obvious choice might seem to be

```
if ( point == lastPoint )
```

Unfortunately, this will *not* work. Testing using `==` will only return `true` if the point objects being compared are actually the same object. Each time `onMouseClicked` is called, the system constructs a new `Location` to describe the cursor position, even if two clicks occur at the same point. Therefore, testing with `==` here would always yield false.

To address such situations, Java objects support a method named `equals`. This method’s purpose is to test if two possibly distinct objects have identical characteristics. In particular, if we construct two distinct `Location` objects that happen to have the same x and y coordinates, the `equals` method will consider them the same and return `true`. Thus, we can complete the `if` statement in the `onMouseClicked` method of our `SteadyTester` by writing

```
if ( point.equals(lastPoint) )
```

4.3 Operators and Precedence Rules

We have already encountered quite a few operators that can be applied to primitive types. In Chapter 2, we presented the four basic arithmetic operators, `+`, `-`, `*`, and `/`. The list of operators was expanded in Chapter 3 with the addition of the relational and logical operators, `<`, `<=`, `>`, `>=`, `==`, `!=`, `&&`, `||`, and `!`.

With this large collection of operators, it is possible to write fairly complex expressions. Therefore, it becomes important to understand the rules Java follows when evaluating such expressions. We introduced two of these rules, Java’s basic *precedence* rules, in Section 2.4. We explained that when evaluating expressions involving just addition, subtraction, multiplication, and division, Java

```

    // A program to detect when two consecutive mouse clicks
    // occur at exactly the same canvas coordinates
public class SteadyTester extends WindowController
{
    // where to display the word "STEADY"
    private static final Location MESSAGE_LOCATION = new Location(100, 100);

    // Text of the message "STEADY"
    private Text steadyMessage;

    // Last position clicked on
    private Location lastPoint = new Location( -1, -1 );

    // Create and hide the "STEADY" message
    public void begin()
    {
        steadyMessage = new Text("STEADY", MESSAGE_LOCATION, canvas );
        steadyMessage.hide();
    }

    // Display "STEADY" if location of click has not changed
    public void mouseClicked( Location point )
    {
        if ( . . . ) { // Need to test that the points are the same
            steadyMessage.show();
        } else {
            steadyMessage.hide();
        }

        lastPoint = point;
    }
}

```

Figure 4.4: A program to test for steady clickers

does multiplications and divisions before additions and subtractions and that it otherwise works from left to right. Now we need to expand these rules to cover the much larger set of operators available.

Java's approach to ordering arithmetics operations should not be new to you. The same rules are followed in standard mathematics. They are more important when programming, however, because programmers have fewer means to make the intended interpretation of a formula clear. When writing a formula on a sheet of paper or using a good word processor we can spread the formula over several lines to make things clear. For example, when we compare the two formulae

$$\frac{height - width}{2}$$

and

$$height - \frac{width}{2}$$

the order in which the operations are to be performed is clear even if one forgets the rule that division should be done before subtraction. When we are forced to write all the symbols in a formula on one line, as we are in Java, it is much easier to accidentally write a formula in which the order of evaluation dictated by the precedence rules is different from the order we had in mind.

The rules for determining the order of arithmetic operators in Java (and mathematics) are based on the notion of assigning operators to precedence levels. Division and multiplication are assigned to one precedence level and addition and subtraction are assigned to a different level. Furthermore, the level for multiplication and division is ranked higher than the level for addition and subtraction. Java performs operations associated with the higher precedence level before those associated with the lower level.

To extend Java's precedence rules to the expanded set of operators we encountered in the last chapter, all we need to know is which operators are assigned to each precedence level and how these levels are ranked. Java assigns the operators we have considered to the seven precedence levels listed below:

- a. Arithmetic and logical negation: `-`, and `!`.
- b. Multiplication, and division: `*` and `/`.
- c. Addition and subtraction: `+` and `-`.
- d. Comparisons: `<`, `<=`, `>`, `>=`.
- e. Equality and inequality: `==`, `!=`
- f. And: `&&`
- g. Or: `||`

The earlier an operator appears in this list, the higher its precedence. That is, Java performs unary operators before multiplications and divisions, multiplications and divisions before additions and subtractions, and so on.

The good news is that Java does not mind if you use extra parentheses. Parentheses can be used to override these precedence rules. Any part of a formula enclosed in parentheses will be

evaluated before its result can be used to perform operations outside the parentheses. Therefore, while you should do your best to become familiar with the precedence rules given above, whenever it will make an expression's meaning clearer, do not hesitate to use parentheses. For example, Java will treat the expressions

```
canvas.getHeight() - canvas.getWidth()/2
```

and

```
canvas.getHeight() - ( canvas.getWidth()/2 )
```

as equivalent. If you find yourself writing an expression like the first example and are not quite sure which operator will be evaluated first, add parentheses to make your intent clear to the computer and to anyone who reads your code later.

Exercise 4.3.1 *What is the value of the following boolean expressions? Assume $x = 3$, $y = -1$ and $z = 2$.*

a. $x * y > -z$

b. $3 * x + 4 * y > 2 * z \ \&\& \ x * y > -z$

c. $!(x + y \leq 1) \ \&\& \ -y * (x + 1) == 2 * z$

d. $3 * x + 4 * y > 2 * z \ \&\& \ x * y > -z \ || \ !(x + y \leq 1) \ \&\& \ -y * (x + 1) == 2 * z$

4.4 Double Time

When we first introduced numeric variables, we mentioned that Java distinguishes between integers and numbers that have fractional components. Java's `int` type corresponds to the integers. A type named `double` is provided for numbers with fractional parts. In this section we will discuss the reasons Java makes this distinction and the impact the distinction has on a programmer when working with numbers.

4.4.1 Java Needs Two Numeric Types

To understand why Java distinguishes integers from other numbers, consider the following two problems. First, suppose you and two companions are stranded in a life boat. Among your supplies, you have 55 gallons of water. You decide that this water should be divided equally among the three of you. How much water does each person get to drink? (Go ahead. Take out your calculator.)

Now, suppose that you are the instructor of a programming course in which 55 students are currently registered and you want to assign the students to 3 afternoon laboratory sections. How many students should be assigned to each section?

With a bit of luck, your answer to the first question was $18 \frac{1}{3}$ gallons. On the other hand, even though the same numbers, 55 and 3, appear in the second problem, "18 $\frac{1}{3}$ students" would probably not be considered an acceptable answer to the second problem (at least not by the student who had to be chopped in thirds to even things out). A better answer would be that there should be two labs of 18 students and a third lab with 19 students.

The point of this example is that there are problems in which fractional results are acceptable and other problems where we know that only integers can be used. If we use a computer to help us solve such problems, we need a way to inform the computer whether we want an integer result or not.

In Java, we do this by choosing to use `ints` or `doubles`. For example, if we declare three instance variables:

```
private double gallons;
private double survivors;
private double waterRation;
```

and then execute the assignment statements

```
gallons = 55;
survivors = 3;
waterRation = gallons/survivors;
```

the number associated with the name `waterRation` will be 18.33333... On the other hand, if we declare the variables

```
private int students;
private int labs;
private int labSize;
```

and then execute the assignments

```
students = 55;
labs = 3;
labSize = students/labs;
```

the number associated with `labSize` will be 18. In the first example, Java can see that we are working with numbers identified as `doubles`, so when asked to do division, it gives the answer as a `double`. In the second case, since Java notices we are using `ints`, it gives us just the integer part of the quotient when asked to do the division.

Of course, the answer we obtain in the second case, 18, isn't quite what we want. If all the labs have exactly 18 students, there will be one student excluded. We would like to do something about such leftovers. Java provides an additional operator with this in mind. When you learned to divide numbers, you probably were at first taught that the result of dividing 55 by 3 was 18 with a remainder of 1. That is, you were taught that the answer to a division problem has two parts, the quotient and the remainder. When working with integers in Java, the `"/"` operator produces the quotient. The percent sign can be used as an operator to produce the remainder. Thus, 55/3 will yield 18 in Java and `55 % 3` will yield 1. In general, `x % y` will only equal 0 if `x` is evenly divisible by `y`.

Used in this way the percent sign is called the *mod* or *modulus* operator. Java assigns this operator to the same precedence level as division and multiplication. We can use this operator to improve our solution to the problem of computing lab sizes by declaring an extra variable

```
private int extraStudents;
```

and adding the assignment

```
extraStudents = students % labs;
```

4.4.2 Arithmetic with doubles and ints

The distinction between `doubles` and `ints` in Java is a feature intended to allow the programmer to control the ways in which arithmetic computations are performed. As a beginning programmer, however, you should be warned that this feature may produce some strange surprises.

Suppose that we want to construct an oval whose width is three quarters of the width of the canvas and we know that the width of the canvas is 300. According to the rules of normal arithmetic, it should not matter whether we say

```
300*3/4
```

or

```
300*(3/4)
```

In Java, however, the first version will produce the expected result, 225, while the second version will produce the number 0. In the first example, Java begins by multiplying 300 and 3 yielding 900 and then divides by 4 to obtain 225. In the second example, Java first divides 3 by 4. Since both of these numbers look like `ints` to Java, it decides the correct answer is 0 with a remainder of 3 and returns the quotient, 0, as the result of the division. Then, 300 times the result of this division, 0, yields 0.

To avoid unexpected results like this, one must understand how Java decides when we desire an integer result and when we would prefer to work with numbers with fractional components.

The first rule is simple. When we write numbers out explicitly, Java decides whether the number is an `int` or a `double` based on whether it contains a decimal point. Therefore, 3 is an `int` but 3.0 is a `double`. This can make a big difference. If we rewrite the second example above as

```
300.0*(3.0/4.0)
```

it will produce 225.0 rather than 0 as its result.

Second, one must understand that while Java distinguishes between `ints` and `doubles`, it recognizes that they are related. In particular, in contexts where one should technically have to provide a `double`, Java will allow you to use an `int`. This was already illustrated in the example above where we declared an instance variable

```
private double gallons;
```

and then wrote the assignment

```
gallons = 55;
```

The number 55 is identified by Java as an `int`. The variable is declared to be a `double`. Normally, Java considers an assignment invalid if the type of the value described by the expression to the right of the equal sign is different from the type of the variable on the left side of the equal sign. The assignments

```
gallons = new Location( 33, 100 );
```

and

```
gallons = true;
```

would be considered illegal because `gallons` is a `double` rather than a `Location` or a `boolean`. Java will, however, accept the assignment

```
gallons = 55;
```

even though it assigns an `int` to a variable that is supposed to refer to a `double`.

Java is willing to convert `ints` into `doubles` because it knows there is only one reasonable way to do the conversion. It simply adds a “.0” to the end of the `int`. On the other hand, Java knows that there are several ways to convert a `double` into an `int`. It could drop the fractional part or it could round. Because it can not tell the correct technique to use without understanding the context or purpose of the program, Java refuses to convert a `double` into an `int` unless explicitly told how to do so using mechanisms that we will discuss later. Therefore, given the instance variable declaration

```
private int students;
```

Java would reject the assignment

```
students = 18.333;
```

as erroneous. It would also reject the assignment

```
students = 55.0;
```

Even if the only digit that appears after the decimal point in a numeric literal is 0, the presence of the decimal point still makes Java think of the number as a `double`.

Java’s willingness to convert `ints` to `doubles` is also a factor in understanding how it decides whether the result of an arithmetic operation should be an `int` or a `double`. If both operands of an arithmetic operator are of the same numeric type, then Java will produce a result of this type. Thus, adding two `ints` produces an `int` and dividing a `double` by another `double` yields a `double`. The interesting question is what does Java do if one operand is an `int` and the other a `double`, as in

3/4.0

The answer is that when Java sees an operator with two operands of different types, it tries to convert one of the operands to the other type. Since it will convert `ints` to `doubles` but not the other way around, if an operation involves one `int` and one `double`, Java converts the `int` into a `double` (by adding .0) and then performs the operation yielding a result that is also a `double`. Accordingly, 3/4.0 would evaluate to 0.75.

The following table summarizes Java’s procedure for determining the type of result an operator should produce given the types of its two operands. The types of the operands determine the row and column of the table that applies. The contents of the selected cell specify the type of the result. Note that the only case where the result produced is an `int` is when both operands are integers.

	int	double
int	int result	double result
double	double result	double result

This means that once you introduce a number which is a `double` into a computation, you are likely to end up with a result that is a `double`. Even if the result has a fractional part that is 0, Java will not automatically convert this `double` into an `int`. This can lead to unexpected errors in your program. For example, several of the accessor methods associated with graphical objects including `getX`, `getY`, `getWidth`, and `getHeight` return values that are `doubles`. Accordingly, the expression in the assignment statement

```
midPointX = ( currentPosition.getX() + previousPosition.getX() ) / 2
```

which might be used to compute the x coordinate of the midpoint of the line between two points on the canvas, would produce a `double`. This means that if you had defined the variable to hold the result as

```
private int midPointX;
```

Java would reject the assignment as an error. Instead, the variable would have to be declared as

```
private double midPointX;
```

Exercise 4.4.1 *What is the value of each of the following expressions?*

- a. `12 / 5`
- b. `35 / 7`
- c. `15.0 / 2.0;`
- d. `65.0 / 4;`
- e. `79 % 12;`

Exercise 4.4.2 *Given that the variable `ratio` is of type `double`, what value will be associated with the `ratio` by each of the following assignments?*

- a. `ratio = 12.0 / 4;`
- b. `ratio = 32 / 6;`
- c. `ratio = 48.0 / 5;`
- d. `ratio = 22 % 8;`

4.4.3 Displaying doubles

Text constructions and `System.out.println`, the two mechanisms we have used to display information on a computer's screen, are flexible enough to display information of many types. In this section, we will discuss the results that are produced when they are used with `double` values.

For small numbers, the only difference between displaying an `int` and displaying a `double` is that Java always includes a decimal point when printing a `double`. If we declare

```
private int anInt;  
private double aDouble;
```

and then associate these names with equivalent values by executing the assignments

```
anInt = 1000;
aDouble = anInt;
```

executing the command

```
System.out.println( anInt + " = " + aDouble );
```

would produce the output

```
1000 = 1000.0
```

Things get a bit more interesting when large values of type `double` are displayed. Suppose that rather than initializing the two variables above to 1000, we instead executed the assignments

```
anInt = 1000000000;
aDouble = anInt;
```

so that both variables had a value of one billion. In this case, the output produced would be

```
1000000000 = 1.0E9
```

The strange item “1.0E9” is an example of Java’s version of scientific notation. Where Java displays 1.0E9, you probably would have expected to see 1000000000.0. This value is one billion or 10^9 . The “E9” in the output Java produces is short for “times ten raised to the power 9”. The “E” stands for “exponent”. In general, to interpret a number output in this form you should raise 10 to the power found after the E and multiply the number before the E by the result. The table below shows some examples of numbers written in this notation and the standard forms of the same values.

E-notation	Standard Representation	Standard Scientific Notation
1.0E8	100,000,000	1×10^8
1.86E5	186,000	1.86×10^5
4.9E-6	.0000049	4.9×10^{-6}

Not only does Java display large numbers using this notation when you use `System.out.println` or `Text` objects, it also recognizes numbers typed in this format as part of a program. So, if you really like scientific notation, you can include a statement like

```
avogadro = 6.022E26;
```

in a Java program.

4.4.4 Why are Rational Numbers Called `double`?

As a final topic in this section, we feel obliged to try to explain why Java chooses to call non-integers `doubles` rather than something like `real` or `rational`. The explanation involves a bit of history and electronics.

To allow us to manipulate numbers in a program, a computer’s hardware must encode the numbers we use in some electronic device. In fact, for each digit of a number there must be a tiny memory device to hold it. Each of these tiny memory devices costs some money and, not long ago,

they cost quite a bit more than they do now. So, if a program is only working with small numbers, the programmer can reduce the hardware cost by telling the computer to set aside a small number of memory devices for each number. On the other hand, when working with larger numbers, more memory devices should be used.

On many machines, the programmer is not free to pick any number of memory devices per number. Instead only two options are available: the “standard” one, and another that provides **double** the number of memory devices per number. The name of the Java type **double** derives from such machines.

While the cost of memory has decreased to the point where we rarely need to worry that using **doubles** might increase the amount of hardware memory our program uses, the name does reflect an important aspect of computer arithmetic. Since each number represented in a computer is stored in physical devices, the total number of digits stored is always limited.

In the computer’s memory, numbers are stored in binary, or base 2. Thirty-one binary digits are used to store the numeric value of each **int**. Just as the number of digits included in a car’s odometer limits the range of mileage values that can be displayed before the odometer flips back to “000000”, the number of binary digits used to represent a number limit the range of numbers that can be stored. As a result, the values that can be processed by Java as **ints** range from -2,147,483,648 (= -2^{31}) to 2,147,483,647 (= $2^{31} - 1$). If you try to assign a number outside this range to an **int** variable, Java is likely to throw away some of the digits yielding an incorrect result. If you need to write a program that works with very large integers, there is another type that is limited to integer values but that can handle numbers with twice as many digits. This type is called **long**. There is also a type named **short** that uses half as much memory as **int** to represent a number. For numbers that are not integers, there is also a type named **float** that is like **double** but uses half as many bits to represent numeric values. The examples in this text will not use **long**, **float**, or **short**.

The range of numbers that can be stored as **double** values is significantly larger than even the **long** type. The largest **double** value is approximately 1.8×10^{308} and the smallest **double** is approximately -1.8×10^{308} . This is because Java stores **double** values as you might write numbers in scientific notation. It rewrites each number as a value, the mantissa, times 10 raised to an appropriate exponent. For example, the number

32,953,923,804,836,184,926,273,582,140,929.584,289

might be written in scientific notation as

$3.2953923804836184926273582140929584289 \times 10^{31}$

Java, however, does not always encode all the digits of the mantissa of a number stored as a **double**. The amount of memory used to store a **double** enables Java to record approximately 15 significant digits of each number. Thus, Java might actually record the number used as an example above as

$3.295392380483618 \times 10^{31}$

This means that if you use numbers with long or repeating sequences of digits, Java will actually be working with approximations of the numbers you specified. The results produced will therefore be slightly inaccurate. Luckily, for most purposes, the accuracy provided by 15 digits of precision is sufficient.

There is one final aspect of the range of `double` values that is limited. First, the smallest number greater than 0 that can be represented as a `double` is approximately 5×10^{-324} . Similarly, the largest number less than 0 that can be represented as a `double` is approximately -5×10^{-324} .

4.4.5 Selecting a Numeric Type

The examples and rules above suggest that a reasonable guideline for dealing with the difference between `ints` and `doubles` is to use `doubles` whenever possible. Most of the surprising examples you are likely to encounter involve integer division. Furthermore, Java will sometimes refuse to accept an assignment of a value to a variable declared as an `int`, but will always allow you to assign any numeric value to a variable declared as a `double`, even if it has to turn an `int` into a `double` to make the assignment possible.

Of course, there are contexts where you will have to use `ints`. First, as suggested in the introduction to this section, there are programming problems where the only acceptable results are integers. You will have to use `ints` in such programs. Finally, there are certain contexts where Java will demand an `int`. The three numbers provided in a `Color` construction, for example, must be `ints`.

4.5 Handy Sources of Numeric Information

Java provides a number of methods in its libraries that can be used to generate useful numerical information. Several of these features are described in this section.

4.5.1 What Time is it?

There are many signs that computers keep track of the time. Most likely, the computer you use displays the current time of day somewhere on your screen as you work. Your computer can tell you when each of your files was created and last modified. While your web browser downloads large files, it probably displays an estimate of how much longer it will take before the process is complete.

To support all the ways in which time is used in programs, Java provides a sophisticated collection of primitives that allow your program to determine the hour, the year, the day of the week or just about anything else you might imagine that is related to measuring time. These mechanisms provide more than you need at this point, but we would like to introduce you to one of these mechanisms, a method named `System.currentTimeMillis`.

The `System.currentTimeMillis` method returns a single number telling you what time it is. For example, if you run a program containing the instruction

```
System.out.println("The time is now " + System.currentTimeMillis() );
```

Java might produce the following mysterious answer:

```
The time is now 1057070809463
```

Apparently, Java uses a different system for telling time than most of us!

Whenever we use a number to answer the question “What time is it?” we are using the number to describe how long it has been since some fixed reference time. If you asked when this book was written and we answered 1422, our answer would appear to be nonsense if you assumed we were

using the Gregorian calendar in which times are measured relative to the year in which Christ was born (at least approximately). In fact, however, the number 1422 describes when our book was written based on the Islamic calendar, which measures time from the migration of the Prophet and his followers from Mecca to Medina.

Java isn't very religious about time, so it measures time from midnight on January 1, 1970. Also, since Java sometimes needs to be very precise about what time it is, Java measures time in milliseconds (i.e., thousandths of a second). The number shown above, 1057070809463, tells you exactly how many milliseconds passed between midnight, January 1, 1970, and the moment at which the Java instruction shown above produced the output

```
The time is now 1057070809463
```

The `System.currentTimeMillis` operation would be very awkward to use if you wanted to display the current time of day in a form a human could understand. Fortunately, it is a very appropriate tool for measuring short intervals of time within a program. This is an ability we will need in many programs in the following chapters. To use `System.currentTimeMillis` to measure an interval, we simply ask Java for the time at the beginning of the interval we need to measure and then again at the end. The length of the interval can then be determined by subtracting the starting time from the ending time.

As an example of how to do such timing, a program that will measure the duration of a click of the mouse button is shown in Figure 4.5. The program first uses `System.currentTimeMillis` in the `onMousePress` method. It assigns the time returned to the variable `startingTime` so that it can access the value after the mouse has been released. In `onMouseRelease`, the program computes the difference between the time at which the mouse was pressed and released. Then, in order to display the result in units of seconds rather than milliseconds, it divides by 1000. A sample of what the program's output might look like is shown in Figure 4.6.

In this program, the variable used to hold the result of invoking `System.currentTimeMillis` is declared as a `double`. In reality, the `System.currentTimeMillis` method does not return its result as a `double`. `System.currentTimeMillis` always returns an integer value as its result, but because these values can become quite large, it uses the type `long` rather than the type `int`. Just as Java is willing to convert an `int` into a `double`, it will convert a `long` into a `double`. It will not, however, convert a `long` into an `int`. Therefore, we must use `double` or `long` variables to hold time values. We suggest using `double` as the type for such variables.

4.5.2 Sines and Wonders

If you review all the things you have learned to do with numbers in Java, you should be unimpressed at best. Think about it. For just \$10 you could go to almost any store that sells office or school supplies and buy a pocket calculator that can compute trigonometric functions, take logarithms, raise any number to any power and do many other complex operations. On the other hand, with Java and a computer that probably costs 100 times as much as a calculator, all you have learned to do so far is add, subtract, divide, and multiply. In this section we will improve this situation by introducing a collection of methods that provide the means to perform more advanced mathematical calculations.

```

// A program to measure the duration of mouse clicks.
public class ClickTimer extends WindowController {

    // location where messages should be displayed
    private final static Location TEXT_POS = Location( 30, 50);

    // When the mouse button was depressed
    private double startingTime;
    // Used to display length of click
    private Text message;

    // Create the Text to display the current count
    public void begin() {
        message = new Text( "Please depress and release the mouse",
                           TEXT_POS, canvas );
    }

    // Record the time that the button is pressed
    public void onMousePress(Location point) {
        startingTime = System.currentTimeMillis();
    }

    // Display the duration of the latest click
    public void onMouseRelease(Location point) {
        message.setText("You held the button down for " +
                       ( System.currentTimeMillis() - startingTime)/1000 +
                       " seconds" );
    }
}

```

Figure 4.5: Java program to measure mouse click duration.



Figure 4.6: Sample output of ClickTimer program

Roots, Logs, and Powers

A common arithmetic operation available on most calculators is the taking of a square root. In Java, this is done using a method named `Math.sqrt`. `Math` is a class in which many interesting mathematical methods have been collected. `Math.sqrt` is a method that takes any numeric value as a parameter and returns a `double` approximating the number's square root. Thus, if we created a `Text` object named `message` and later executed the instruction

```
message.setText("The square root of 2 is " + Math.sqrt(2) );
```

the message

```
The square root of 2 is 1.4142135623730951
```

would appear on the canvas.

Of course, the parameter to a method like `Math.sqrt` can be described using any expression that produces a numeric result and the invocation of `Math.sqrt` can be used as a sub-part of a larger arithmetic expression. For example, if the variable names `a`, `b`, and `c` were associated with the coefficients of a quadratic equation

$$ax^2 + bx + c = 0$$

we could compute a solution to the equation by translating the quadratic formula (simplified to produce only the largest answer):

$$\frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

into Java. The result would look like:

```
( -b + Math.sqrt( b*b - 4*a*c) ) / (2 * a)
```

Note that all the parentheses included in this Java fragment are required to ensure that the operations specified are performed in the desired sequence.

Java provides several other methods for performing operations related to raising values to powers:

`Math.pow(a,b)` raises the number `a` to the power `b` and returns the result,

`Math.exp(b)` raises the constant e (≈ 2.718) to the power `b`,

`Math.ln( a )` returns the natural logarithm (i.e., the logarithm base e) of the number `a`.

In addition, if you need to work with the number e and don't feel like looking it up and typing in enough digits to approximate it accurately, the name `Math.E` is already associated with a very accurate approximation to e .

Trigonometric Functions

Another important set of functions provided in Java through the `Math` class are the standard trigonometric functions: sine, cosine, and tangent and their inverses. The Java names for these functions are shown in the table below:

Java	Function
<code>Math.sin(x)</code>	returns the sine of x
<code>Math.cos(x)</code>	returns the cosine of x
<code>Math.tan(x)</code>	returns the tangent of x
<code>Math.asin(x)</code>	returns the arc sine of x
<code>Math.acos(x)</code>	returns the arc cosine of x
<code>Math.atan(x)</code>	returns the arc tangent of x

For all of these methods, Java assumes that the angles involved are measured in radians rather than degrees. The `Math` class contains features to make it a bit easier for those who prefer degrees to work with radians. There are two methods named `Math.toRadians` and `Math.toDegrees` that can be used to convert from one set of units to the other. Also, the name `Math.PI` is associated with a very accurate approximation of the mathematical constant π . Thus, the cosine of a right angle would be computed by saying:

```
Math.cos( Math.toRadians( 90 ) )
```

or

```
Math.cos( Math.PI / 2 )
```

Converting doubles into ints

There are situations where you need to convert a `double` into an `int`. In such situations, there are two main techniques you can use. If you have a `double` and you want the integer value nearest this double, then you should provide the `double` as a parameter to the method `Math.round`. The result produced will be the `int` obtained by rounding the `double` to the nearest integer value. Thus, given the declarations

```
private double aDouble;
private int anInt;
```

the assignments

```
aDouble = 4.7;
anInt = Math.round(aDouble);
```

will set `anInt` equal to 5.

If instead you just want to discard any fractional component of the `double`, you can convert the `double` into an `int` using an operation called a *type cast*. A type cast is formed by placing the name of the desired type of the result, `int` in our case, in parentheses before an expression that describes the value to be converted. Given the declarations of `anInt` and `aDouble` shown above, the assignments

```
aDouble = 4.7;
anInt = (int ) aDouble;
```

will set `anInt` equal to 4.

Absolute Values

The method `Math.abs` can be used to compute the absolute value of a number. This method accepts `int` or `double` values and produces a result that is the same type as its parameters. The expressions

```
Math.abs( 17 )
```

and

```
Math.abs( -17 )
```

both yield the `int` value 17, while the expressions

```
Math.abs( -34.2 )
```

and

```
Math.abs( 34.2 )
```

both yield the `double` value 34.2.

The `Math.abs` method is sometimes used to test whether `double` values are nearly equal. If you test to see whether two `double` values are exactly equal using `==`, the test may yield `false` even when the values being compared should be equal. This occurs because the computer only records about 15 digits of any `double` value. For example, although by definition

$$a = \sqrt{a} \times \sqrt{a}$$

the Java expression

```
3 == Math.sqrt(3.0)*Math.sqrt(3.0)
```

will evaluate to `false`. Since `Math.sqrt(3.0)` produces a value that only approximates $\sqrt{3}$, the product `Math.sqrt(3.0)*Math.sqrt(3.0)` evaluates to 2.9999999999999996.

To deal with such inaccuracies, it is often better to test whether two values are approximately equal rather than exactly equal. If `a` and `b` are two `double` values, then a test of the form

```
if ( Math.abs( a - b ) < EPSILON )
```

where `EPSILON` is a constant with an appropriately small value, is a simple way to test for approximate equality.

4.6 Strings

By now it should be clear that Java programs can manipulate many kinds of data and that the language classifies the different kinds of data it can manipulate into types. Also, it is undeniably clear that one of the most important types of data that computers process is text. Text based programs such as word processors and e-mail applications are among the most important and widely used applications on personal computers. Therefore, it should not be a surprise that Java has a type for manipulating textual data. This type is named `String`.

We have been using `String` values since our very first example program in Figure 1.1. Quoted pieces of text like the message

`"I'm Touched"`

which appeared in that first program are literals describing values of the `String` type.

We have also seen that Java provides an operator for manipulating `Strings`. The operator is the plus sign. When used with `Strings` it denotes concatenation, the operation of joining two sequences of symbols together, rather than addition. Thus, in examples like

```
curDisplay.setText("You have clicked " + theCount + " times.");
```

we are instructing Java to concatenate the digits of the value of `theCount` onto the end of the text “You have clicked ” and then to tack the text “ times.” onto the end of the result of this first concatenation.

The fact that Java considers `String` a type implies that we can do several things with `Strings` that Java allows us to do with any type. We can use `Strings` as parameters. We have done this in both `Text` constructors and invocations of the `System.out.println` method. We can also define instance variables of `String` type and use assignment statements to associate values with these variables.

To illustrate the potential value of using `String` variables, consider the program shown in Figure 4.7. This program implements a simple interface one might use to practice encoding information in Morse code.

In Morse code, each letter of the alphabet is encoded using a sequence of long and short signals. These signals are called dashes and dots because a standard way to represent the signals on paper is to draw a dash for a long signal and a dot for a short signal. Thus,

. . . - - - . . .

represents a message composed of three short signals followed by three long signals followed by three short signals. This happens to be the Morse code for the distress signal “SOS”.

The program in Figure 4.7 translates short and long presses of the computer’s mouse button into a sequences of dots and dashes. If someone running the program used the mouse to enter the sequence discussed above, the program would display the output shown in Figure 4.8 The idea is that a person trying to learn Morse code could use the mouse to practice signaling and then look at the sequence of dots and dashes the program displays on the screen to see if they got it right. This means that we don’t need to worry about which sequences of dots and dashes go with which letters of the alphabet to write the program. All we need to know about Morse code is that a short signal is a dot and a long signal is a dash.

The program uses a `String` variable named `currentCode` to keep track of the message it should display on the screen. Initially, the display will simply be

`Code =`

Then, each time the user clicks the mouse, the program will add a dot or a dash to the message. If the first thing the user does is a short click, the message will become

`Code = .`

If this is followed by two long clicks, the message will become

`Code = . - -`

```

    // A program to display Morse code entered by pressing
    // the mouse as dots and dashes on the canvas.
public class MorseCode extends WindowController {

    // Location where code entered is displayed
private static final Location DISPLAY_POS = new Location( 30, 30);
    // Minimum time (in milliseconds) for a dash
private static final double DASH_TIME = 200;

    // String to hold sequence entered so far
private String currentCode = "Code = ";
    // Text used to display sequence on canvas
private Text display;

    // Time when mouse was last depressed
private double pressTime;

    // Create display
public void begin()
{
    display = new Text("Code = ", DISPLAY_POS, canvas);
}

    // Record time at which mouse was depressed
public void onMousePress ( Location point)
{
    pressTime = System.currentTimeMillis();
}

    // Add a dot or dash depending on click length
public void onMouseRelease( Location point)
{
    if ( System.currentTimeMillis() - pressTime > DASH_TIME )
    {
        currentCode = currentCode + " -";
    }
    else
    {
        currentCode = currentCode + " .";
    }
    display.setText( currentCode );
}
}

```

Figure 4.7: A program to display Morse code as dots and dashes

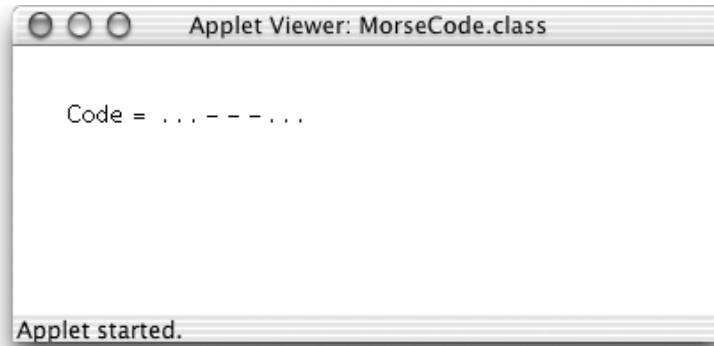


Figure 4.8: Sample of Morse code program display

and so on.

To add a dash to the value of the variable `currentCode`, the program uses an assignment statement of the form

```
currentCode = currentCode + " -";
```

which can be found in `onMouseRelease`. This statement looks like and acts a great deal like the similar assignment we have used to increment integer variables

```
count = count + 1;
```

It takes the existing text associated with `currentCode`, adds a dash to it, and then tells Java to make this the new value of `currentCode`.

The program uses the `System.currentTimeMillis` method described in Section 4.5.1 to access the current time when the mouse is pressed and released. It compares the time that elapses between these two events to a value, `DASH.TIME`. If the elapsed time is greater than this, it executes the assignment shown above to add a dash. Otherwise it uses a similar assignment to add a dot. Either way, after the mouse is released it uses `setText` to update the `Text` object named `display` to hold the extended sequences of dots and dashes.

Everything we have told you about the `String` type so far would suggest it is a primitive type. Java provides literals for describing its values and allows us to use the `+` operator to manipulate `String` values. In fact, however, `String` is actually a class rather than a primitive type. It is a very special class. It is the only class with which Java provides literals.

One way to tell that `String` is a class is that Java also provides methods for manipulating `String` values. For now, the most important of these methods is the `equals` method. As with other objects, when we want to compare two `Strings` to see if they are the same, it is usually not appropriate to use `==`. Instead, the `equals` method should be used. For example, if we wanted to check that the code entered in our program was the sequence for SOS, we might say

```
if ( currentCode.equals("Code = . . . - - - . . .") )
{ . . .
```

Java provides many other accessor methods to operate on `Strings`. For example,

```
currentCode.length()
```

will produce an `int` value telling how many characters long a `String` is, and

```
if (currentCode.endsWith(". . . - - - . . .") )  
{ . . .
```

could be used to check to see if someone using our program entered a sequence that ended with the SOS signal. We will consider these functions and other aspects of using `Strings` in more detail in Chapter ??.

4.7 Chapter Review Problems

Exercise 4.7.1 *Define the following terms:*

- a. *primitive values*
- b. *primitive types*
- c. `long`
- d. *modulus*

Exercise 4.7.2 *Assuming a program contained the two declarations:*

```
private Location myLocation = new Location (10, 10);  
private Location yourLocation = new Location (10, 10);
```

- a. *What result would be produced by the expression*

```
myLocation == yourLocation
```

- b. *What result would be produced by the expression*

```
myLocation.equals(yourLocation)
```

- c. *What coordinates would describe the `Locations` associated with the two variable names after the following statements had been executed?*

```
myLocation.translate( 15, 15);  
yourLocation.translate( -5, -5);
```

Exercise 4.7.3 *Assuming a program contained the two declarations:*

```
private Location myLocation = new Location (10, 10);  
private Location yourLocation = myLocation;
```

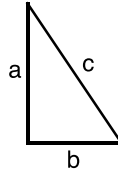


Figure 4.9: A right-angle triangle

- a. What result would be produced by the expression

```
myLocation == yourLocation
```

- b. What result would be produced by the expression

```
myLocation.equals(yourLocation)
```

- c. What coordinates would describe the `Locations` associated with the two variable names after the following statements had been executed?

```
myLocation.translate( 15, 15);
yourLocation.translate( -5, -5);
```

Exercise 4.7.4 Let's revisit a classic in the world of mathematics, the Pythagorean theorem. The Pythagorean theorem states that for a right-angle triangle like that in Figure 4.9, $a^2 + b^2 = c^2$. Assume that `a`, `b`, and `c` have been declared as variables of type `double`. Which one of the following expressions accurately describes the length of side `a`? What do the other two expressions represent?

- a. `Math.sqrt(c * c) - Math.sqrt(b * b)`
- b. `Math.sqrt(Math.pow(c, 2)) + Math.sqrt(Math.pow(b, 2))`
- c. `Math.sqrt(c * c - b * b)`

Exercise 4.7.5 Given that the variable `books` is of type `int` and the variable `length` is of type `double`, what value will be associated with the variable given the following assignments?

- a. `books = 250 * (4 / 5);`
- b. `length = 660 / 22.0;`
- c. `books = 660 / 22.0;`
- d. `length = 485.5 / 7.3;`
- e. `books = 250.0 * (4.0 / 5.0 );`

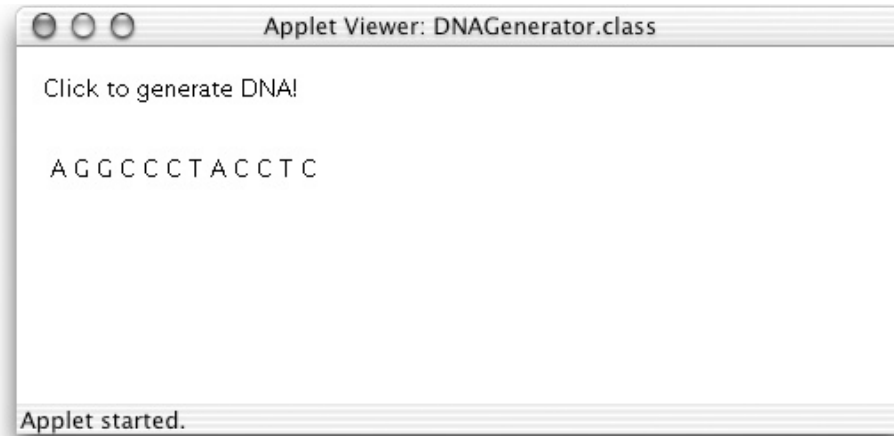


Figure 4.10: Display for DNAGenerator

f. `books = 660 / 22;`

g. `length = 21 * 7;`

h. `length = 443 / 17;`

Exercise 4.7.6 Fill in the condition for the following code. `myNumber` has been declared as an `int`.

```
if ( // myNumber is evenly divisible by either 3 or 4 ) {  
    // do something  
}
```

Exercise 4.7.7 Assuming `lastTime` has been declared to be of type `double` and then been assigned a value previously using `System.currentTimeMillis`, what is wrong with the following line of code? How would you fix it?

```
System.out.println("It has been " + ( System.currentTimeMillis() - lastTime )  
                  + " seconds since last time");
```

4.8 Programming Problems

Exercise 4.8.1 Write a program called `ClickTimer2` that displays the time between mouse clicks. For simplicity, assume that the time when the program starts to run counts as a click as well.

Exercise 4.8.2

- a. DNA stands for deoxyribonucleic acid and is a very long polymer of four different types of nucleotide bases. These four bases are Guanine(G), Adenine (A), Thymine (T), and Cytosine(C). It is the sequence of these bases that uniquely determine DNA. Write a simple

program called `DNAGenerator` that may impress any friends of yours with interests in biology. This program will generate a strand of DNA by randomly adding one of the the four letters “G”, “A”, “T” or “C” each time the mouse is clicked. It also displays the generated DNA strand as shown in Figure 4.10.

- b. Modify your program so that no more than 20 bases are added to the DNA strand.

Exercise 4.8.3 In Morse code, a single dot is used to represent the letter “E”, a single dash represents a “T”, and the letter “D” is represented by a dash followed by two dots. This means that the sequence

- . .

has two possible interpretations. It could be a single letter “D” or it could be the word “TEE”.

The lengths of the pauses between dots and dashes are used to resolve such ambiguities. If the pause between a pair of dots is short, then the two signals are treated as part of a single letter. If the pause is long, the signals are treated as parts of separate letters. To make it possible to represent such long and short pauses in our graphical representation of sequences of dots and dashes we might represent the sequence for “D” as

- . .

and the sequence for “TEE” as

-

That is, we would put more spaces between pairs of symbols if the pause between them was longer. Thus, the signal for “TED” would look like

- . . - . .

Modify `MorseCode` so that it adds extra spaces for long pauses. Assume a pause is long if its duration equals (or exceeds) that of a dash.